ByteMart Digital

THE COMPLETE GUIDE

Spring Boot

From Fundamentals to Professional Practice

A complete, example-driven guide to Spring Boot — from IoC, REST APIs, and data access to security, production readiness, testing, and containerised deployment.

30	200+	14	8
Chapters	Code Examples	Visual Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Spring Boot: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided “as is”.

Trademarks
Spring and Spring Boot are trademarks of Broadcom Inc. and/or its subsidiaries. All other product names, logos, and brands are the property of their respective owners and are used for identification purposes only.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. What Is Spring Boot? 4
2. IoC & Dependency Injection 7
3. Beans & the Application Context 10
4. Auto-Configuration & Starters 13
5. REST Controllers 16
6. Request Mapping & Parameters 19
7. Request & Response Bodies 22
8. Validation & Error Handling 25
9. Spring Data JPA Basics 28
10. Entities & Relationships 31
11. Repositories & Queries 34
12. Transactions 37
13. Externalized Configuration 40
14. Profiles & Environments 43
15. Logging 46
16. Spring Security Fundamentals 49
17. Authentication 52
18. Authorization & JWT 55
19. Actuator & Health 58
20. Metrics & Observability 61
21. Caching 64
22. Scheduling & Async 67
23. Unit Testing 70
24. Integration Testing 73
25. Testing Web Layers 76
26. Testing Data Layers 79
27. AOP & Cross-Cutting Concerns 82
28. Building & Packaging 85
29. Docker & Deployment 87
30. Best Practices & Architecture 90

MODULE 1 · SPRING BOOT FOUNDATIONS

CHAPTER 01

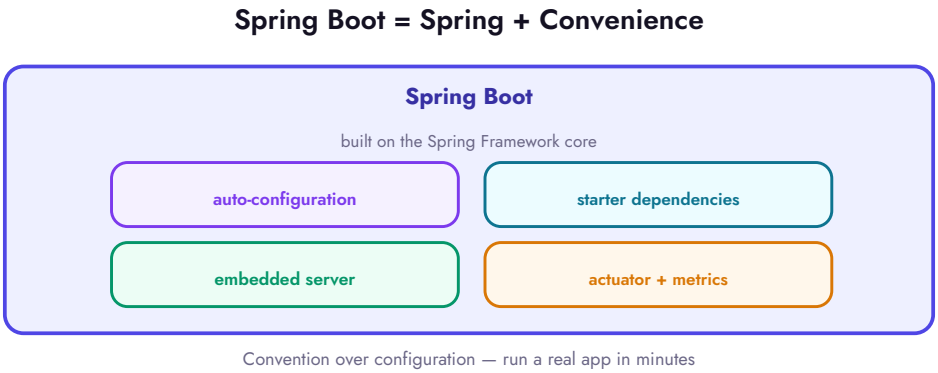
What Is Spring Boot?

How Spring Boot removes Spring's configuration burden with opinionated defaults and starters.

Spring is a powerful but historically verbose framework. **Spring Boot** sits on top of it and removes the ceremony: sensible defaults, dependency bundles, and an embedded server let you build a real, production-ready application in minutes rather than days of XML.

> **`Spring Boot`**
> An opinionated layer over the Spring Framework that auto-configures beans, bundles dependencies as starters, and embeds a server — so you can run a production-grade app with almost no boilerplate.

What Boot adds to Spring



Spring Boot builds on Spring with auto-configuration, starters, an embedded server, and actuator.

- **Auto-configuration** — configures beans based on what is on the classpath.
- **Starters** — curated dependency bundles like `spring-boot-starter-web`.
- **Embedded server** — Tomcat/Jetty built in; no separate deployment.
- **Actuator** — production endpoints for health and metrics out of the box.

A complete application

A single annotated class is a runnable Spring Boot application. `@SpringBootApplication` enables component scanning, auto-configuration, and configuration in one.

```java
@SpringBootApplication
public class ShopApplication {
    public static void main(String[] args) {
        SpringApplication.run(ShopApplication.class, args);
    }
}
// mvn spring-boot:run  -> a running web server on port 8080
```

The project layout

Boot projects follow a convention: source under `src/main/java`, configuration in `application.properties` or `application.yml`, and a build file (`pom.xml` or `build.gradle`) declaring starters.

TIP

Generate a new project at **start.spring.io** (Spring Initializr). Pick your dependencies and it produces a ready-to-run, conventionally-structured project — the standard way every team begins.

Convention over configuration

Spring Boot keeps the full power of Spring while removing the configuration burden. Add a starter, and Boot wires up sensible defaults you can override anytime — letting you focus on business logic, not plumbing.

© 2026 ByteMart Digital

MODULE 1 · SPRING BOOT FOUNDATIONS

CHAPTER 02

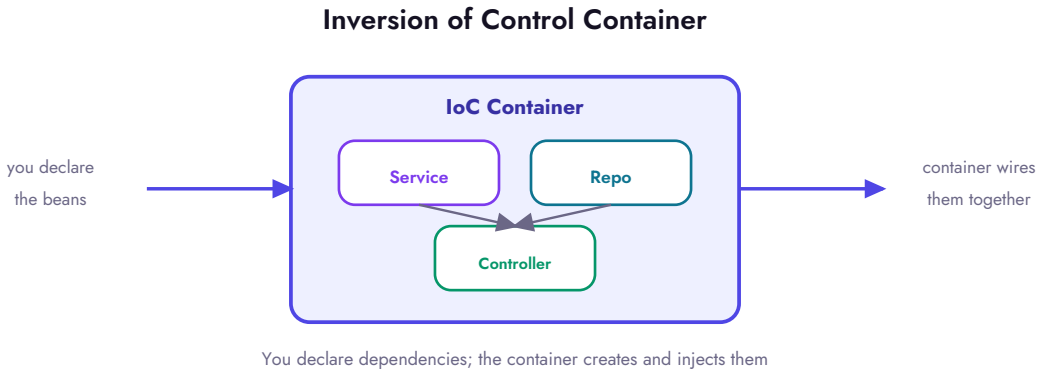
IoC & Dependency Injection

The core idea behind Spring — letting the container create and wire your objects.

Everything in Spring rests on one principle: **Inversion of Control**. Instead of your code creating its own dependencies, a container creates them and injects them where needed. This **dependency injection** makes code loosely coupled, testable, and easy to change.

Inversion of Control

Normally an object constructs what it needs. With IoC, the **container** owns that responsibility — it builds objects and supplies their dependencies. Your class just declares what it requires.



You declare dependencies; the IoC container creates and injects them.

Constructor injection

The preferred form: declare dependencies as constructor parameters. Spring supplies them automatically, and the fields can be `final`.

```java
@Service
public class OrderService {
    private final PaymentGateway gateway;
    public OrderService(PaymentGateway gateway) {   // injected
        this.gateway = gateway;
    }
}
```

Why DI matters

- **Loose coupling** — depend on interfaces, not concrete classes.
- **Testability** — inject mocks or fakes in tests easily.
- **Flexibility** — swap implementations without touching callers.
- **No manual wiring** — the container assembles the object graph.

© 2026 ByteMart Digital

Constructor vs field injection

WATCH OUT

Prefer constructor injection over `@Autowired` on fields. It makes dependencies explicit, allows `final` fields, and lets you construct the object in tests without Spring at all.

Let the container wire it

Dependency injection hands object creation and wiring to Spring, leaving your classes to declare what they need. The result is loosely-coupled, testable code — the foundation everything else in Spring builds on.

© 2026 ByteMart Digital

MODULE 1 · SPRING BOOT FOUNDATIONS

CHAPTER 03

Beans & the Application Context

03

What beans are, how Spring discovers them, and the lifecycle the container manages.

A **bean** is any object the Spring container manages. The **application context** is the container that creates beans, wires them together, and manages their lifecycle. Understanding how beans are found and created explains how a Spring application assembles itself.

> **Bean**
> An object instantiated, assembled, and managed by the Spring IoC container, typically discovered by component scanning or declared in configuration.

Declaring beans

Stereotype annotations mark classes as beans for component scanning. Each is a specialised `@Component` that also conveys the class's role.

Annotation	Role
@Component	generic bean
@Service	business logic
@Repository	data access
@Controller / @RestController	web endpoints

@Bean methods

For objects you do not own (or need to configure), declare them with `@Bean` methods inside a `@Configuration` class.

```java
@Configuration
public class AppConfig {
    @Bean
    public RestClient restClient() {
        return RestClient.builder().baseUrl("https://api.x").build();
    }
}
```

The bean lifecycle

The container instantiates a bean, injects its dependencies, runs initialisation callbacks, keeps it ready for use, and calls destruction callbacks at shutdown.

© 2026 ByteMart Digital

Bean Lifecycle



@PostConstruct runs after injection; @PreDestroy before removal

The container manages the whole lifecycle for you

The container instantiates, injects, initialises, uses, and finally destroys each bean.

java

```
@Component
public class Cache {
    @PostConstruct void warmUp()  { /* after injection */ }
    @PreDestroy   void flush()   { /* before shutdown */ }
}
```

Managed objects in a container

Beans are objects Spring creates and wires; the application context is the container that manages them from construction to destruction. Component scanning and `@Bean` methods are how you populate it.

MODULE 1 · SPRING BOOT FOUNDATIONS

CHAPTER 04

Auto-Configuration & Starters

How Boot configures your application automatically based on the classpath — and how to override it.

Auto-configuration is Spring Boot's defining trick. Rather than making you configure a data source, web server, or JSON mapper, Boot detects what is on the classpath and configures sensible defaults — which you can override the moment you need to.

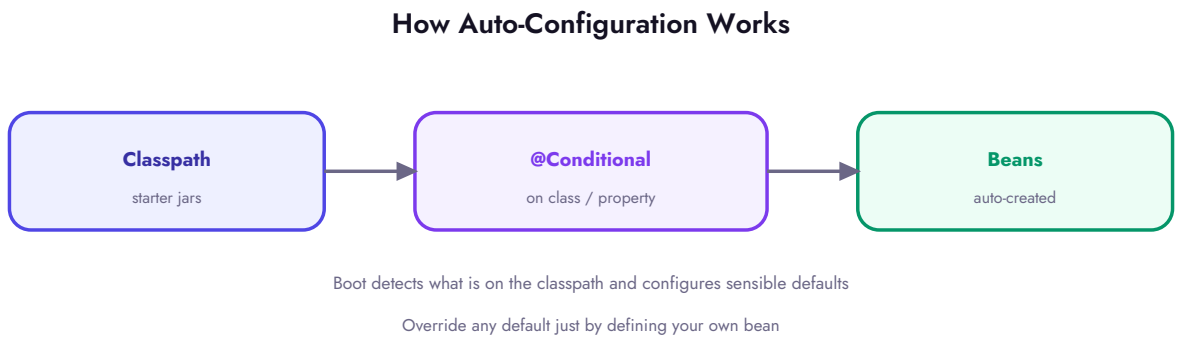
Starters

A **starter** is a curated set of dependencies for a capability. Adding one pulls in everything needed and triggers the matching auto-configuration.

Starter	Brings
spring-boot-starter-web	REST, embedded Tomcat, JSON
spring-boot-starter-data-jpa	JPA, Hibernate, transactions
spring-boot-starter-security	authentication, authorization
spring-boot-starter-test	JUnit, Mockito, test support

How auto-configuration decides

Auto-configuration classes are guarded by **conditions**: create this bean only if a class is present, or a property is set, or no such bean already exists. This is why defaults appear automatically yet never overwrite yours.



Boot inspects the classpath and conditionally creates the beans your app needs.

Overriding a default

To replace an auto-configured bean, simply define your own — the `@ConditionalOnMissingBean` guard steps aside for it.

```java
@Bean
public ObjectMapper objectMapper() {
    return new ObjectMapper()          // your own replaces the default
        .findAndRegisterModules()
        .disable(SerializationFeature.FAIL_ON_EMPTY_BEANS);
}
```

Seeing what was configured

TIP

Run with `--debug` to print the auto-configuration report — it lists which auto-configurations matched and why. Invaluable when Boot configured something unexpected, or failed to.

Sensible defaults, always overridable

Starters declare a capability and auto-configuration wires it up conditionally. You get a working setup for free, and full control the instant you define your own bean — convenience without lock-in.

© 2026 ByteMart Digital

MODULE 2 · BUILDING WEB APIS

CHAPTER 05

REST Controllers

Exposing HTTP endpoints with @RestController and mapping them to methods.

A REST controller turns Java methods into HTTP endpoints. Spring routes each incoming request to the right method, converts the return value to JSON, and sets the response — letting you focus on logic, not plumbing.

A first controller

java

```java
@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService service;
    public ProductController(ProductService service) { this.service = service; }

    @GetMapping
    public List<Product> all() { return service.findAll(); }
}
```

`@RestController` combines `@Controller` and `@ResponseBody`, so return values are serialised to the response body (JSON by default).

The Request Lifecycle



DispatcherServlet routes the request through your layered application

The DispatcherServlet routes each request through controller, service, and repository.

The HTTP verb mappings

Annotation	Verb / use
@GetMapping	read
@PostMapping	create
@PutMapping	replace
@PatchMapping	partial update
@DeleteMapping	remove

A full CRUD controller

java

```java
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Product create(@RequestBody Product p) {
    return service.save(p);
}

@DeleteMapping("/{id}")
public void delete(@PathVariable Long id) {
    service.delete(id);
}
```

TIP

Keep controllers thin: parse the request, call a service, shape the response. Business logic belongs in the service layer, not the controller — this keeps endpoints testable and reusable.

Methods become endpoints

A REST controller maps HTTP verbs and paths to methods and serialises results to JSON. Keep it a thin translation layer over your services, and your API stays clean and maintainable.

© 2026 ByteMart Digital

MODULE 2 · BUILDING WEB APIS

CHAPTER 06

Request Mapping & Parameters

Extracting data from the URL, query string, headers, and path with binding annotations.

Real endpoints need input: an id in the path, filters in the query string, a token in a header. Spring binds each of these to method parameters automatically, converting types as it goes.

Path variables

java

```
@GetMapping("/{id}")
public Product one(@PathVariable Long id) {
    return service.findById(id);        // /api/products/42
}
```

Query parameters

`@RequestParam` binds query-string values. Provide defaults and mark optional ones so missing parameters do not fail the request.

java

```
@GetMapping
public Page<Product> search(
        @RequestParam(defaultValue = "") String q,
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size) {
    return service.search(q, page, size);   // ?q=shoes&page=1
}
```

Headers and the body

Annotation	Binds
@PathVariable	a segment of the URL path
@RequestParam	a query-string parameter
@RequestHeader	an HTTP header value
@RequestBody	the deserialised request body

java

```
@GetMapping("/me")
public User me(@RequestHeader("Authorization") String auth) {
    return service.fromToken(auth);
}
```

© 2026 ByteMart Digital

Type conversion

Spring converts request strings to the parameter type — `Long`, `LocalDate`, enums, and more. A bad value yields a 400 automatically.

NOTE

For many related parameters, bind them into an object with `@ModelAttribute` or a record instead of a long parameter list — it reads better and validates as a unit.

Bind input declaratively

Spring maps path segments, query parameters, headers, and the body to typed method parameters, converting as needed. Declare what you want and Spring extracts it — no manual parsing.

MODULE 2 · BUILDING WEB APIS

CHAPTER 07

Request & Response Bodies

Serialising and deserialising JSON with Jackson, DTOs, and response shaping.

Modern APIs speak JSON. Spring Boot uses Jackson to convert between JSON and Java objects automatically — request bodies become objects, and returned objects become JSON. Shaping this conversion well keeps your API clean and stable.

Automatic JSON binding

java

```java
@PostMapping
public OrderResponse create(@RequestBody OrderRequest req) {
    // req is deserialised from JSON automatically
    Order saved = service.place(req);
    return OrderResponse.from(saved);   // serialised back to JSON
}
```

Use DTOs, not entities

Expose **Data Transfer Objects** at the API boundary rather than JPA entities. DTOs decouple your API contract from your database schema and prevent leaking internal fields.

java

```java
public record ProductDto(Long id, String name, String priceLabel) {
    static ProductDto from(Product p) {
        return new ProductDto(p.getId(), p.getName(),
            "Rs " + p.getPrice());
    }
}
```

Shaping JSON with Jackson

- `@JsonProperty("name")` — rename a field in the JSON.
- `@JsonIgnore` — omit a field entirely.
- `@JsonInclude(NON_NULL)` — drop null fields from output.
- `@JsonFormat` — control date and number formatting.

ResponseEntity for full control

Return `ResponseEntity` when you need to set the status, headers, or body explicitly rather than relying on defaults.

© 2026 ByteMart Digital

```java
@GetMapping("/{id}")
public ResponseEntity<ProductDto> one(@PathVariable Long id) {
    return service.find(id)
        .map(p -> ResponseEntity.ok(ProductDto.from(p)))
        .orElse(ResponseEntity.notFound().build());
}
```

WATCH OUT

Never expose JPA entities directly in responses. Lazy associations can trigger errors during serialisation, and you risk leaking internal or sensitive fields. Map to a DTO first.

JSON in, JSON out — via DTOs

Spring and Jackson handle JSON conversion automatically; your job is to expose clean DTOs, not entities. That separation keeps your API contract stable and independent of your database.

© 2026 ByteMart Digital

MODULE 2 · BUILDING WEB APIS

CHAPTER 08

Validation & Error Handling

Validating input with Bean Validation and returning consistent error responses.

Never trust incoming data. **Bean Validation** checks request bodies declaratively, and a global exception handler turns failures — and any other error — into consistent, well-structured responses. Together they make your API robust and predictable.

Declaring constraints

java

```java
public record CreateUser(
    @NotBlank String name,
    @Email String email,
    @Size(min = 8) String password,
    @Min(18) int age) { }
```

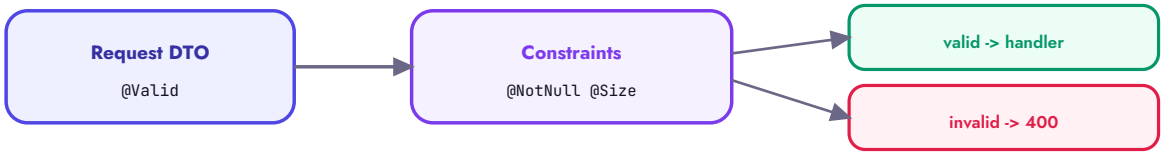
Triggering validation

Add `@Valid` to the parameter. If any constraint fails, Spring throws before your method runs, and the failure becomes a 400 response.

java

```java
@PostMapping
public User create(@Valid @RequestBody CreateUser req) {
    return service.register(req);
}
```

Bean Validation Flow



Constraints are checked automatically; failures become 400 responses

Constraints are checked automatically; valid requests proceed, invalid ones become 400s.

Global exception handling

A `@RestControllerAdvice` class centralises error handling, turning exceptions into a uniform error body across every controller.

```java
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse onInvalid(MethodArgumentNotValidException e) {
        return ErrorResponse.of("VALIDATION", e.getMessage());
    }
    @ExceptionHandler(NotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse onMissing(NotFoundException e) {
        return ErrorResponse.of("NOT_FOUND", e.getMessage());
    }
}
```

Consistent error shape

- Return the same error structure everywhere (code, message, details).
- Map each exception type to the right HTTP status.
- Never leak stack traces or internal messages to clients.
- Log the full error server-side; return a safe summary to the caller.

Validate at the edge, fail cleanly

Bean Validation rejects bad input before it reaches your logic, and a global advice turns every failure into a consistent response. Robust APIs validate declaratively and handle errors in one place.

© 2026 ByteMart Digital

MODULE 3 · DATA ACCESS

CHAPTER 09

Spring Data JPA Basics

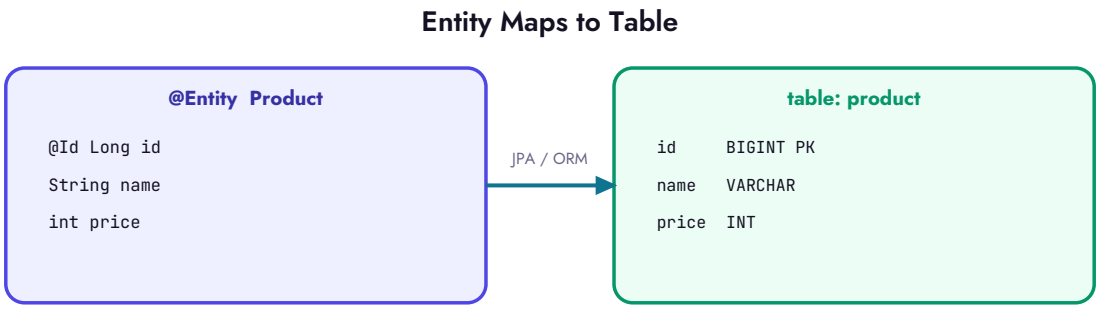
Persisting objects to a relational database with entities and repositories — almost no SQL.

Spring Data JPA lets you store and query objects in a relational database with barely any boilerplate. You map classes to tables with annotations and declare repository interfaces — Spring generates the implementation, transactions, and most queries for you.

Mapping an entity

An `@Entity` class maps to a table; its fields map to columns. `@Id` marks the primary key, and `@GeneratedValue` lets the database assign it.

```java
@Entity
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private int price;
    // getters / setters
}
```



JPA maps an entity class and its fields to a table and columns.

The repository

Extend `JpaRepository` and you inherit `save`, `findById`, `findAll`, `delete`, paging, and sorting — with no implementation to write.

```java
public interface ProductRepository
        extends JpaRepository<Product, Long> { }

// injected and used:
productRepository.save(product);
Optional<Product> p = productRepository.findById(1L);
List<Product> all = productRepository.findAll();
```

What JPA and Hibernate do

- **ORM** — translate between objects and rows automatically.
- **JPA** is the specification; **Hibernate** is the default implementation.
- Generate SQL for CRUD, so you rarely write it by hand.
- Track changes to managed entities and flush them on commit.

TIP

For local development, add the H2 in-memory database and set `spring.jpa.hibernate.ddl-auto=update` so tables are created from your entities — you can run and iterate with zero database setup.

Objects, not SQL

Spring Data JPA maps entities to tables and gives you full CRUD from an empty interface. You describe the shape of your data and let the framework handle persistence — dropping to SQL only when you truly need to.

© 2026 ByteMart Digital

MODULE 3 · DATA ACCESS

CHAPTER 10

Entities & Relationships

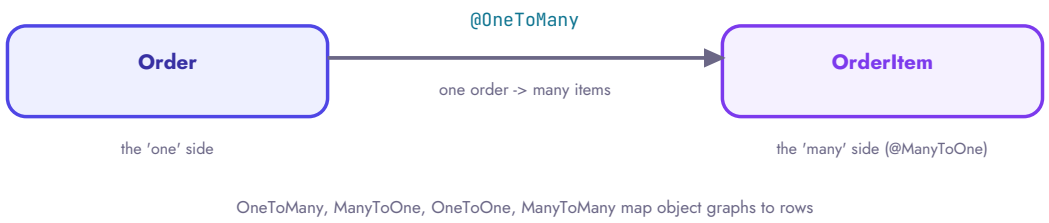
Modelling associations between entities and controlling how they load.

Real data is connected: an order has items, a user has roles. JPA maps these **relationships** between entities to foreign keys and join tables. Getting the mappings and their loading behaviour right is central to a healthy data layer.

The four relationship types

Annotation	Models
@OneToMany / @ManyToOne	one owner, many children
@OneToOne	a one-to-one link
@ManyToMany	a join table between both

Entity Relationships



OneToMany, ManyToOne, OneToOne, and ManyToMany map object graphs to rows.

A bidirectional OneToMany

java

```
@Entity
public class Order {
    @Id @GeneratedValue Long id;
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL)
    private List<OrderItem> items = new ArrayList<>();
}

@Entity
public class OrderItem {
    @Id @GeneratedValue Long id;
    @ManyToOne
    @JoinColumn(name = "order_id")
    private Order order;      // the owning side (has the FK)
}
```

Lazy vs eager loading

Associations load **lazily** (on first access) or **eagerly** (with the parent). Collections default to lazy; loading them outside a transaction causes the classic `LazyInitializationException`.

- Keep collections **lazy** — eager loading pulls data you may not need.
- Load what you need with a **fetch join** or an `@EntityGraph`.
- Watch for the **N+1 query** problem when iterating associations.
- The **owning side** holds the foreign key; the other uses `mappedBy`.

WATCH OUT

The N+1 problem: loading a list, then a related collection per element, fires one query plus N more. Detect it with SQL logging and fix it with a fetch join or entity graph.

Model links, control loading

Relationship annotations map object graphs to foreign keys and join tables. Keep associations lazy, fetch deliberately, and watch for N+1 — relationships are where data-layer performance is won or lost.

© 2026 ByteMart Digital

MODULE 3 · DATA ACCESS

CHAPTER 11

Repositories & Queries

Deriving queries from method names, and writing custom JPQL or native queries when needed.

Spring Data can write queries for you from method names, and lets you drop to JPQL or SQL when the query is complex. This spectrum — derived, declared, and native — covers almost every data-access need with minimal code.

Derived query methods

Name a method after the query you want and Spring generates it. Keywords like `findBy`, `And`, `OrderBy`, and `Containing` compose into real queries.

java

```
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByNameContainingIgnoreCase(String q);
    List<Product> findByPriceLessThanOrderByPriceAsc(int max);
    Optional<Product> findBySku(String sku);
    long countByActiveTrue();
}
```

Custom JPQL with @Query

When a name would get unwieldy, write the query explicitly with `@Query`. JPQL works on entities and fields, not tables and columns.

java

```
@Query("select p from Product p where p.price between :lo and :hi")
List<Product> inPriceRange(@Param("lo") int lo, @Param("hi") int hi);
```

Native queries

For database-specific SQL or performance-critical statements, use a native query with `nativeQuery = true`.

java

```
@Query(value = "SELECT * FROM product WHERE MATCH(name) AGAINST(?1)",
       nativeQuery = true)
List<Product> fullTextSearch(String term);
```

Paging and projections

- Return `Page<T>` and accept a `Pageable` for paginated results.
- Use **projections** (interfaces or records) to select only needed columns.
- `@Modifying` plus `@Query` runs bulk update or delete statements.

© 2026 ByteMart Digital

- Sort with a `Sort` argument or an `OrderBy` in the method name.

From method names to native SQL

Spring Data covers a spectrum: derive simple queries from method names, declare complex ones with JPQL, and drop to native SQL when required. Reach for the simplest option that expresses the query clearly.

MODULE 3 · DATA ACCESS

CHAPTER 12

Transactions

Grouping database operations into all-or-nothing units with @Transactional.

A **transaction** groups several database operations so they succeed or fail together — essential for consistency. Spring makes this declarative: annotate a method `@Transactional` and Spring manages the commit and rollback around it.

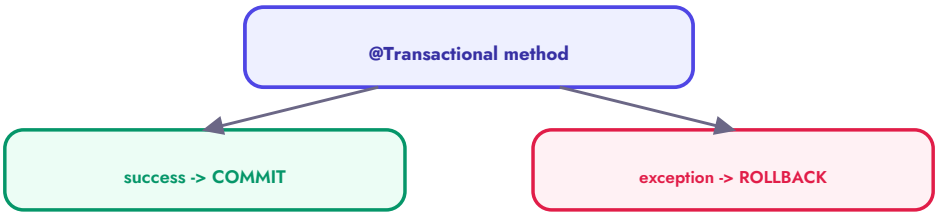
> **Transaction**
> A unit of work that either fully commits or fully rolls back, keeping the database in a consistent state even when something fails midway.

Declarative transactions

java

```java
@Service
public class TransferService {
    @Transactional
    public void transfer(Long from, Long to, int amount) {
        accounts.debit(from, amount);
        accounts.credit(to, amount);   // if this throws,
    }                                  // the debit is rolled back too
}
```

@Transactional: Commit or Roll Back



All writes commit together, or none do — propagation controls nesting

On success the transaction commits; on a runtime exception it rolls back.

Rollback rules

By default Spring rolls back on unchecked (runtime) exceptions and commits on checked ones. Override with `rollbackFor` when you need different behaviour.

java

```java
@Transactional(rollbackFor = PaymentException.class)
public void checkout(Cart cart) throws PaymentException { }
```

Propagation

Propagation controls what happens when a transactional method calls another. `REQUIRED` (the default) joins the existing transaction; `REQUIRES_NEW` suspends it and starts a fresh one.

Propagation	Behaviour
REQUIRED	join or start one (default)
REQUIRES_NEW	always a new transaction
SUPPORTS	join if one exists, else none
MANDATORY	must already be in one

WATCH OUT

Spring transactions work through proxies, so a `@Transactional` method called from within the same class is not intercepted. Call it through the injected bean, or move it to another bean.

All or nothing, declaratively

`@Transactional` wraps a method in a database transaction that commits on success and rolls back on failure. Understand rollback rules and propagation, and mind the proxy caveat — then consistency is nearly automatic.

MODULE 4 · CONFIGURATION & PROFILES

CHAPTER 13

Externalized Configuration

Keeping settings out of code with properties, YAML, environment variables, and type-safe binding.

Configuration — database URLs, API keys, feature flags — should live outside your code so the same build runs in every environment. Spring Boot reads settings from many sources and binds them into your beans, type-safely.

application.properties / yml

```text
# application.yml
server:
  port: 8080
app:
  currency: INR
  max-cart-items: 50
  payment:
    api-key: change-me
```

Reading a single value

```java
@Value("${app.currency}")
private String currency;   // injected from configuration
```

Type-safe configuration

For groups of related settings, bind them into a `@ConfigurationProperties` record or class. This is type-safe, validated, and far cleaner than scattering `@Value` everywhere.

```java
@ConfigurationProperties(prefix = "app")
public record AppProps(String currency, int maxCartItems,
                       Payment payment) {
    public record Payment(String apiKey) { }
}
// inject AppProps anywhere and read props.currency()
```

The property precedence

Boot layers configuration sources, with later ones overriding earlier. This lets you set defaults in the JAR and override them per environment without rebuilding.

- Command-line arguments (highest priority).

- Environment variables — ideal for secrets in containers.
- External `application.yml` next to the JAR.
- Packaged `application.yml` inside the JAR (defaults).

WATCH OUT

Never commit secrets (API keys, passwords) to source control. Supply them through environment variables or a secrets manager, and keep only safe defaults in the packaged configuration.

Config outside the code

Externalised configuration lets one build behave differently per environment. Prefer type-safe `@ConfigurationProperties` over scattered values, and inject secrets from the environment — never from source.

MODULE 4 · CONFIGURATION & PROFILES

CHAPTER 14

Profiles & Environments

Running the same application differently across dev, test, and production with profiles.

One codebase must run in many environments — a developer's laptop, a test server, production. **Profiles** let you define environment-specific configuration and beans, and activate the right set at startup without changing code.

Profile-specific configuration

Files named `application-{profile}.yml` load only when that profile is active, layering over the base configuration.

```text
# application-dev.yml       -> H2, verbose logging
# application-prod.yml      -> real DB, INFO logging
spring:
  datasource:
    url: jdbc:postgresql://db:5432/shop
```

Activating a profile

```bash
java -jar app.jar --spring.profiles.active=prod
# or via env var:
SPRING_PROFILES_ACTIVE=prod java -jar app.jar
```

Profile-specific beans

Annotate a bean with `@Profile` to include it only in certain environments — a fake payment gateway in dev, the real one in prod.

```java
@Bean @Profile("dev")
PaymentGateway fakeGateway() { return new FakeGateway(); }

@Bean @Profile("prod")
PaymentGateway realGateway() { return new StripeGateway(key); }
```

Good profile practice

- Keep shared settings in the base file; put only differences in profile files.
- Common profiles: `dev`, `test`, `staging`, `prod`.
- Activate `prod` explicitly — never rely on it being the default.

© 2026 ByteMart Digital

- Tests can set their own profile with `@ActiveProfiles`.

One build, many environments

Profiles select environment-specific configuration and beans at startup, so a single artifact runs correctly everywhere. Keep differences small and explicit, and activate the target profile deliberately.

© 2026 ByteMart Digital

MODULE 4 · CONFIGURATION & PROFILES

CHAPTER 15

Logging

Recording what your application does with SLF4J, log levels, and structured output.

15

Logs are your window into a running application. Spring Boot configures logging out of the box through SLF4J and Logback — you just add log statements and tune levels. Good logging is the difference between diagnosing an issue in minutes and guessing for hours.

Adding logs

Use SLF4J's `Logger` and its parameterised messages — they avoid building strings unless the level is active.

```java
private static final Logger log = LoggerFactory.getLogger(OrderService.class);

log.info("Placing order {} for user {}", orderId, userId);
log.warn("Low stock for product {}", sku);
log.error("Payment failed", exception);   // pass the exception last
```

Log levels

Level	Use for
ERROR	failures needing attention
WARN	unexpected but handled
INFO	key business events
DEBUG	detailed developer info
TRACE	very fine-grained detail

Configuring levels

```text
logging:
  level:
    root: INFO
    com.bytemart.shop: DEBUG
    org.hibernate.SQL: DEBUG   # see generated SQL
```

Logging in production

- Prefer **structured (JSON) logs** so log aggregators can parse them.
- Include a **correlation id** per request to trace it across services.
- Never log secrets, tokens, or full personal data.

© 2026 ByteMart Digital

- Log business events at INFO; keep DEBUG for non-production.

See what your app is doing

SLF4J plus Boot's defaults give you logging with no setup; your job is to log the right things at the right level. Parameterised, structured, secret-free logs turn production debugging from guesswork into a quick read.

© 2026 ByteMart Digital

MODULE 5 · SECURITY

CHAPTER 16

Spring Security Fundamentals

How Spring Security protects an application through a chain of servlet filters.

Spring Security is the standard way to secure a Spring application. It works as a chain of servlet **filters** that intercept every request before it reaches your controllers, handling authentication (who you are) and authorization (what you may do).

The filter chain

Every request passes through the security filter chain. Filters establish the caller's identity, then check whether that identity is allowed to reach the requested endpoint.

The Security Filter Chain



Every request passes through filters that authenticate, then authorize

Rejected requests never reach your controller

Requests pass through filters that authenticate and authorize before reaching a controller.

Authentication vs authorization

Term	Answers
Authentication	who are you?
Authorization	what may you do?

Configuring security

A `SecurityFilterChain` bean declares which endpoints are public, which require authentication, and how users log in.

```java
@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/public/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

Password storage

WATCH OUT

Never store passwords in plain text. Encode them with a strong adaptive hash — Spring Security's `BCryptPasswordEncoder` is the standard choice — and compare hashes, never raw values.

Security as a filter chain

Spring Security intercepts every request through filters that authenticate and authorize before your code runs. You declare the rules in a `SecurityFilterChain`; the framework enforces them consistently.

© 2026 ByteMart Digital

MODULE 5 · SECURITY

CHAPTER 17

Authentication

Verifying who a user is — loading users, encoding passwords, and the authentication flow.

Authentication establishes identity. Spring Security provides the machinery — you supply how to look up a user and how passwords are encoded, and it handles verifying credentials and populating the security context.

Loading users

Implement `UserDetailsService` to load a user by username from your database. Spring calls it during authentication and compares the encoded password.

java

```
@Service
public class DbUserDetailsService implements UserDetailsService {
    private final UserRepository users;
    public UserDetails loadUserByUsername(String username) {
        User u = users.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(username));
        return org.springframework.security.core.userdetails.User
            .withUsername(u.getUsername())
            .password(u.getPasswordHash())
            .roles(u.getRoles().toArray(String[]::new))
            .build();
    }
}
```

Encoding passwords

java

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
// at registration: user.setPasswordHash(encoder.encode(rawPassword));
```

The authentication flow

- The user submits credentials (form, basic auth, or a token).
- A filter builds an `Authentication` request.
- The `AuthenticationManager` loads the user and checks the password.
- On success the authenticated principal is stored in the security context.

Reading the current user

```java
@GetMapping("/me")
public String me(@AuthenticationPrincipal UserDetails user) {
    return user.getUsername();
}
```

Prove identity, safely

Authentication in Spring Security means supplying a user lookup and a password encoder; the framework verifies credentials and populates the security context. Always store hashed passwords and read the principal from context, never from the request.

MODULE 5 · SECURITY

CHAPTER 18

Authorization & JWT

Controlling access by role and securing stateless APIs with JSON Web Tokens.

Once a user is authenticated, **authorization** decides what they may do. For stateless REST APIs, **JWTs** carry identity in a signed token instead of a server session — the standard pattern for modern, scalable services.

Role-based authorization

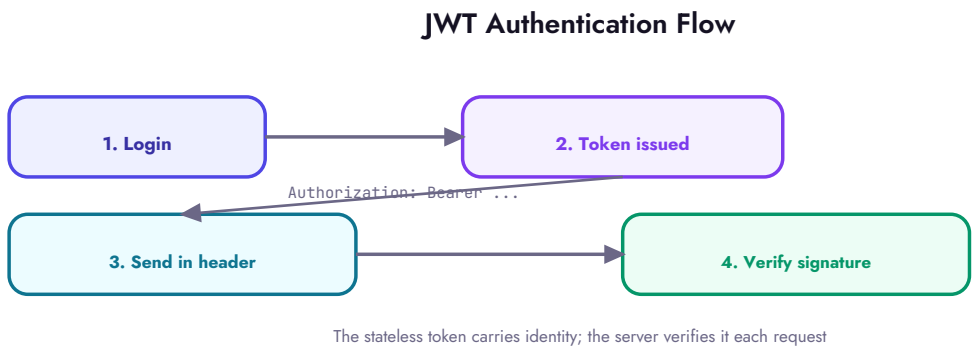
Restrict endpoints by role in the security configuration, or method-by-method with `@PreAuthorize` when the rule depends on the specific call.

```java
@PreAuthorize("hasRole('ADMIN')")
public void deleteProduct(Long id) { }

@PreAuthorize("#userId == authentication.name")
public Profile getProfile(String userId) { }   // owner-only
```

Why JWT for APIs

A **JSON Web Token** is a signed, self-contained token holding the user's identity and roles. The server verifies its signature on each request — no session to store, so any instance can handle any request.



Login issues a signed token; the client sends it each request and the server verifies it.

The token structure

- **Header** — the signing algorithm.
- **Payload** — claims: subject, roles, expiry.
- **Signature** — verifies the token was not tampered with.
- Sent as `Authorization: Bearer <token>`.

Validating on each request

java

```java
// A filter extracts and verifies the token, then sets the context
String token = header.substring(7);            // after 'Bearer '
Claims claims = jwtService.verify(token);      // throws if invalid
var auth = new UsernamePasswordAuthenticationToken(
    claims.getSubject(), null, rolesFrom(claims));
SecurityContextHolder.getContext().setAuthentication(auth);
```

WATCH OUT

Keep JWTs short-lived and always verify the signature and expiry. Never put secrets in the payload — it is signed, not encrypted, so anyone can read its contents.

Authorize by role, scale with tokens

Authorization gates actions by role or ownership; JWTs carry verified identity so APIs stay stateless and horizontally scalable. Sign tokens, keep them short-lived, and validate every request.

© 2026 ByteMart Digital

MODULE 6 · PRODUCTION-READY

CHAPTER 19

Actuator & Health

Exposing operational endpoints for health, info, and readiness with Spring Boot Actuator.

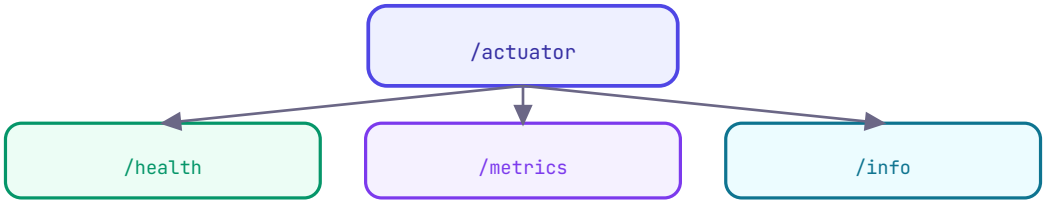
Running in production means answering 'is it healthy?' and 'what is it doing?'. **Spring Boot Actuator** adds ready-made operational endpoints — health, info, metrics, and more — that monitoring systems, load balancers, and orchestrators rely on.

Enabling Actuator

Add the starter and endpoints appear under `/actuator`. Expose only what you need, and secure them.

```text
# add spring-boot-starter-actuator, then:
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
  endpoint:
    health:
      show-details: when-authorized
```

Actuator: Production Endpoints



Built-in endpoints expose health, metrics, and app info for ops

Actuator exposes health, metrics, and info endpoints for operations tooling.

Health checks

`/actuator/health` aggregates checks for the database, disk space, and more into an overall status — exactly what a load balancer or Kubernetes probe queries.

```text
GET /actuator/health
{ "status": "UP",
  "components": { "db": {"status": "UP"}, "diskSpace": {"status": "UP"} } }
```

Liveness and readiness

- **Liveness** — is the app alive? Failing it triggers a restart.
- **Readiness** — can it serve traffic yet? Failing it removes it from the pool.
- Boot exposes both as health groups for Kubernetes probes.
- Custom `HealthIndicator` beans add your own checks.

java

```
@Component
public class QueueHealth implements HealthIndicator {
    public Health health() {
        return queue.isConnected() ? Health.up().build()
                                   : Health.down().build();
    }
}
```

WATCH OUT

Actuator endpoints can reveal sensitive internals. Expose only the endpoints you need, secure them behind authentication, and never open `/actuator` fully to the public internet.

Built-in operability

Actuator gives you the health, readiness, and info endpoints operations teams expect, with almost no code. Expose them deliberately and secure them — they are the interface between your app and the platform running it.

MODULE 6 · PRODUCTION-READY

CHAPTER 20

Metrics & Observability

Measuring application behaviour with Micrometer metrics, and the three pillars of observability.

You cannot improve what you cannot see. **Observability** — metrics, logs, and traces — lets you understand a running system's behaviour. Spring Boot integrates **Micrometer** to collect metrics and export them to systems like Prometheus.

The three pillars

Pillar	Answers
Metrics	how much / how fast (aggregates)
Logs	what happened (discrete events)
Traces	where time went across services

Built-in metrics

Actuator plus Micrometer expose JVM memory, GC, thread, HTTP request, and datasource metrics automatically at `/actuator/metrics` and in a Prometheus-scrapable format.

Custom metrics

Add your own business metrics with a `MeterRegistry` — counters for events, timers for durations, gauges for current values.

java

```
@Service
public class OrderService {
    private final Counter placed;
    public OrderService(MeterRegistry reg) {
        this.placed = reg.counter("orders.placed");
    }
    public void place(Order o) {
        // ...
        placed.increment();          // business metric
    }
}
```

Distributed tracing

In a multi-service system, tracing follows a single request across services with a shared trace id. Micrometer Tracing propagates it and exports spans to tools like Zipkin or Tempo.

- Export metrics to **Prometheus**; visualise in **Grafana**.
- Correlate logs, metrics, and traces by a request/trace id.

- Track the golden signals: latency, traffic, errors, saturation.
- Alert on symptoms users feel, not just internal counters.

Make behaviour visible

Metrics, logs, and traces together explain how a system behaves in production. Boot and Micrometer collect them with little effort — add business metrics, export to your stack, and alert on what users actually experience.

© 2026 ByteMart Digital

MODULE 6 · PRODUCTION-READY

CHAPTER 21

Caching

Speeding up expensive operations with Spring's declarative caching abstraction.

Caching stores the result of an expensive operation so repeated calls return instantly. Spring's caching abstraction makes this declarative — annotate a method and its results are cached, with a pluggable backing store like Caffeine or Redis.

Enabling caching

java

```
@Configuration
@EnableCaching
public class CacheConfig { }   // plus a cache provider on the classpath
```

Caching a method

`@Cacheable` returns a cached value if present, otherwise runs the method and stores the result. The arguments form the cache key.

java

```
@Cacheable("products")
public Product findById(Long id) {
    return repository.findById(id).orElseThrow();   // runs only on a miss
}
```

Keeping the cache fresh

Annotation	Effect
@Cacheable	store / return cached result
@CachePut	always run, update the cache
@CacheEvict	remove entries on change

java

```
@CacheEvict(value = "products", key = "#p.id")
public Product update(Product p) { return repository.save(p); }
```

Caching wisely

- Cache data that is **read often** and **changes rarely**.
- Set a **TTL** so stale data eventually refreshes.

© 2026 ByteMart Digital

- Evict or update on writes to avoid serving stale results.
- Use a distributed cache (Redis) when multiple instances share data.

WATCH OUT

Cache invalidation is famously hard. A cache that is never evicted serves stale data; one evicted too aggressively provides no benefit. Match TTL and eviction to how the data actually changes.

Fast reads, declaratively

Spring's caching turns an annotation into a fast path for expensive reads, backed by the store you choose. Cache the right data, keep it fresh with eviction and TTLs, and reach for a distributed cache when instances must agree.

© 2026 ByteMart Digital

MODULE 6 · PRODUCTION-READY

CHAPTER 22

Scheduling & Async

Running background jobs on a schedule and offloading work to run asynchronously.

Not all work happens inside a request. Spring makes it easy to run **scheduled** jobs — nightly reports, cleanup tasks — and to run slow work **asynchronously** so a request can return without waiting.

Scheduled tasks

Enable scheduling, then annotate methods with `@Scheduled` using a fixed rate, fixed delay, or cron expression.

java

```java
@Configuration @EnableScheduling
public class SchedulingConfig { }

@Component
public class Reports {
    @Scheduled(cron = "0 0 2 * * *")   // 2 AM daily
    public void nightlyReport() { /* ... */ }

    @Scheduled(fixedRate = 60000)      // every 60s
    public void refreshCache() { /* ... */ }
}
```

Asynchronous methods

`@Async` runs a method on a separate thread, returning immediately (or a `CompletableFuture`). Ideal for sending email or calling a slow external service without blocking the request.

java

```java
@Configuration @EnableAsync
public class AsyncConfig { }

@Service
public class Notifier {
    @Async
    public void sendWelcomeEmail(User u) {
        // runs off the request thread
    }
}
```

Using a proper executor

- Configure a bounded `ThreadPoolTaskExecutor` — do not rely on the default.
- For scheduled jobs across many instances, use a distributed scheduler or lock.
- Handle exceptions in async methods — they do not propagate to the caller.
- Keep scheduled jobs idempotent in case they overlap or retry.

© 2026 ByteMart Digital

WATCH OUT

Like transactions, `@Async` works through a proxy: calling an async method from within the same class runs it synchronously. Invoke it through the injected bean instead.

Work beyond the request

`@Scheduled` runs jobs on a timer and `@Async` offloads slow work to background threads. Configure real executors, keep jobs idempotent, and mind the proxy rule — then background processing is straightforward.

© 2026 ByteMart Digital

MODULE 7 · TESTING

CHAPTER 23

Unit Testing

Testing a single class in isolation with JUnit and Mockito — fast, focused, no Spring context.

The fastest, most numerous tests are **unit tests**: they exercise one class in isolation, with its dependencies replaced by mocks. Because they do not start Spring, they run in milliseconds — the base of a healthy test suite.

A plain unit test

Constructor injection pays off here: create the class directly, passing mocks, with no Spring context at all.

```java
class PricingServiceTest {
    @Test
    void appliesDiscount() {
        PricingService service = new PricingService(0.1);   // 10%
        assertEquals(90, service.finalPrice(100));
    }
}
```

Mocking dependencies

When a class depends on a collaborator, use Mockito to supply a fake with scripted behaviour, then verify the interaction.

```java
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock PaymentGateway gateway;
    @InjectMocks OrderService service;

    @Test
    void chargesOnCheckout() {
        when(gateway.charge(anyInt())).thenReturn(true);
        service.checkout(cart);
        verify(gateway).charge(cart.total());   // interaction verified
    }
}
```

What to unit-test

- Business logic, calculations, and edge cases.
- Branching and error paths — not just the happy path.
- One behaviour per test, named for what it verifies.
- Anything you can test without a database or network.

© 2026 ByteMart Digital

TIP

Design classes so their logic can be tested without Spring. If a unit test needs the whole application context to run, that is usually a sign the class is doing too much or is too coupled.

Fast, isolated, plentiful

Unit tests verify one class with mocked collaborators and no Spring context, so they run instantly. They should be the majority of your suite — covering logic and edge cases the cheapest way possible.

MODULE 7 · TESTING

CHAPTER 24

Integration Testing

Verifying that components work together by loading the Spring context with @SpringBootTest.

Unit tests prove pieces work alone; **integration tests** prove they work together. `@SpringBootTest` loads the Spring context so real beans are wired as they are in production — catching configuration and wiring problems units cannot.

A full-context test

java

```
@SpringBootTest
class OrderFlowTest {
    @Autowired OrderService orderService;
    @Autowired OrderRepository orders;

    @Test
    void placesAndPersistsOrder() {
        Order o = orderService.place(sampleRequest());
        assertTrue(orders.findById(o.getId()).isPresent());
    }
}
```

Test databases

Integration tests need a database. Use an in-memory H2 for speed, or — for fidelity — a real database in a container via **Testcontainers**, which spins one up for the test and tears it down after.

java

```
@SpringBootTest
@Testcontainers
class RepositoryIT {
    @Container
    static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16");
    // Spring points the datasource at the container
}
```

Replacing beans with mocks

Use `@MockitoBean` to swap a real bean (like an external payment gateway) for a mock inside the loaded context — real wiring, controlled boundaries.

java

```
@SpringBootTest
class CheckoutIT {
    @MockitoBean PaymentGateway gateway;   // mock only the external edge
}
```

- Test real wiring, transactions, and configuration together.
- Mock only true external boundaries (payment, email, third-party APIs).
- Prefer Testcontainers over H2 when database behaviour matters.
- Keep them fewer than unit tests — they are slower.

Prove it works assembled

Integration tests load the real context so beans, transactions, and configuration are exercised together. They catch wiring and config bugs — keep them focused and fewer, mocking only the genuine external edges.

MODULE 7 · TESTING

CHAPTER 25

Testing Web Layers

Testing controllers and HTTP behaviour with MockMvc and web-slice tests.

Controllers deserve their own tests: routing, status codes, JSON shape, and validation. `@WebMvcTest` loads just the web layer and gives you `MockMvc` to fire simulated HTTP requests without starting a server.

A web-slice test

`@WebMvcTest` loads only the specified controller and web infrastructure — fast, and focused on HTTP concerns. Service dependencies are mocked.

```java
@WebMvcTest(ProductController.class)
class ProductControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean ProductService service;

    @Test
    void returnsProduct() throws Exception {
        when(service.find(1L)).thenReturn(new Product(1L, "Pen", 50));
        mvc.perform(get("/api/products/1"))
           .andExpect(status().isOk())
           .andExpect(jsonPath("$.name").value("Pen"));
    }
}
```

Testing input and errors

```java
@Test
void rejectsInvalidBody() throws Exception {
    mvc.perform(post("/api/users")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"email\":\"bad\"}"))
       .andExpect(status().isBadRequest());
}
```

What web tests verify

- Correct **status codes** for success, not-found, and validation.
- Request routing and parameter binding.
- The **JSON shape** of responses (via `jsonPath`).
- Security rules on endpoints, when combined with security test support.

© 2026 ByteMart Digital

Full end-to-end

For a real HTTP round trip through the whole stack, use `@SpringBootTest(webEnvironment = RANDOM_PORT)` with `TestRestTemplate` or `WebTestClient`.

Test the HTTP contract

`@WebMvcTest` and MockMvc verify routing, status codes, JSON, and validation without a running server. They pin down your API's HTTP contract cheaply — reserve full-server tests for true end-to-end checks.

MODULE 7 · TESTING

CHAPTER 26

Testing Data Layers

Verifying entities, repositories, and queries with @DataJpaTest.

The data layer needs its own tests: do entities map correctly, do derived queries return the right rows, do custom queries work? `@DataJpaTest` loads just the JPA slice — repositories, entities, and a test database — for fast, focused persistence tests.

A repository test

`@DataJpaTest` configures an in-memory database, scans entities and repositories, and rolls back each test automatically.

java

```java
@DataJpaTest
class ProductRepositoryTest {
    @Autowired ProductRepository repo;

    @Test
    void findsByNameFragment() {
        repo.save(new Product(null, "Blue Pen", 50));
        repo.save(new Product(null, "Red Pencil", 30));

        List<Product> hits = repo.findByNameContainingIgnoreCase("pen");
        assertEquals(2, hits.size());
    }
}
```

Testing custom queries

Custom `@Query` methods are exactly what data tests should cover — they contain logic the framework did not generate for you.

java

```java
@Test
void returnsOnlyInRange() {
    // given products at 10, 50, 90
    List<Product> mid = repo.inPriceRange(20, 60);
    assertEquals(1, mid.size());          // only the 50 product
}
```

Good data-test practice

- Each test is transactional and **rolled back** — no leftover data.
- Seed only the rows the test needs; keep it readable.
- Prefer Testcontainers when a query uses database-specific SQL.
- Assert on results, not on the generated SQL.

© 2026 ByteMart Digital

TIP

Test derived query methods and custom queries, but do not test the framework itself — there is no value in asserting that a plain `save` and `findById` work. Focus on your own query logic.

Verify persistence, not the framework

`@DataJpaTest` loads a fast JPA slice with a test database and rolls back each test. Use it to prove your mappings and queries behave — the parts you wrote, not Spring Data's built-ins.

© 2026 ByteMart Digital

MODULE 8 · ADVANCED & DEPLOYMENT

CHAPTER 27

AOP & Cross-Cutting Concerns

Applying behaviour like logging, timing, and auditing across many methods with aspects.

Some concerns — logging, timing, security checks, transactions — cut across many methods and do not belong in business logic. **Aspect-Oriented Programming** lets you define that behaviour once and apply it declaratively, keeping your core code clean.

> **Aspect**
> A module that encapsulates a cross-cutting concern and weaves its behaviour into selected methods without modifying their code.

How Spring AOP works

Spring creates a proxy around a bean and runs your **advice** before, after, or around matching methods. This is the very mechanism behind `@Transactional`, `@Cacheable`, and `@Async`.

A timing aspect

java

```java
@Aspect @Component
public class TimingAspect {
    @Around("@annotation(Timed)")           // pointcut
    public Object time(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.nanoTime();
        try { return pjp.proceed(); }        // run the real method
        finally {
            long ms = (System.nanoTime() - start) / 1_000_000;
            log.info("{} took {} ms", pjp.getSignature(), ms);
        }
    }
}
```

The vocabulary

Term	Meaning
Aspect	the cross-cutting module
Advice	code to run (before/after/around)
Pointcut	which methods it applies to
Join point	a specific matched method call

When to use AOP

- Good for logging, timing, auditing, and metrics.

© 2026 ByteMart Digital

- Keeps repetitive concerns out of business methods.
- Remember it works via proxies — self-calls are not advised.
- Do not hide essential logic in aspects — keep behaviour discoverable.

Behaviour, applied broadly

AOP factors cross-cutting concerns into aspects woven in by proxies — the same technique powering `@Transactional` and `@Cacheable`. Use it for logging and timing, but keep core logic in plain sight.

MODULE 8 · ADVANCED & DEPLOYMENT

CHAPTER 28

Building & Packaging

Producing a runnable, self-contained JAR — and structuring it for fast container builds.

Spring Boot packages your entire application — code, dependencies, and an embedded server — into a single runnable JAR. No external server to install, no WAR to deploy: `java -jar app.jar` and it runs anywhere.

The executable JAR

The Spring Boot build plugin produces a **fat JAR** containing your classes plus every dependency, with a launcher that runs your main class.

bash

```
mvn clean package          # -> target/app-0.0.1.jar
java -jar target/app-0.0.1.jar
# starts the embedded server on port 8080
```

Overriding configuration at runtime

The same JAR adapts to its environment through external configuration and profiles — no rebuild needed.

bash

```
java -jar app.jar \
  --spring.profiles.active=prod \
  --server.port=9000
```

Layered JARs for Docker

A **layered JAR** separates rarely-changing dependencies from your fast-changing code, so Docker can cache the dependency layers and rebuild only your code — dramatically faster image builds.

- Dependencies change rarely — cache them in their own layer.
- Application code changes often — keep it in the top layer.
- Result: fast rebuilds and small image diffs on each deploy.
- Boot's build plugin can generate an optimised image directly.

TIP

Use the build plugin's image goal (`mvn spring-boot:build-image`) to produce an optimised, layered container image with no hand-written Dockerfile — it applies good defaults out of the box.

One artifact, runs anywhere

Boot builds a self-contained executable JAR that runs with a single command and adapts via external config. Layer it for container builds, and the same artifact flows from laptop to CI to production unchanged.

© 2026 ByteMart Digital

MODULE 8 · ADVANCED & DEPLOYMENT

CHAPTER 29

Docker & Deployment

Containerising a Spring Boot app and the practices that make deployments reliable.

Containers package your application with its runtime so it behaves identically everywhere. A Spring Boot JAR drops naturally into a container image, which then runs on any platform — a developer's machine, CI, or a Kubernetes cluster.

A simple Dockerfile

```text
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/app-0.0.1.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

From Code to Running Container



Boot packages everything into one runnable JAR, ready to containerise

Same artifact runs on your laptop, CI, and production

Source builds to a fat JAR, then a container image, then runs the same everywhere.

Configuring containers

Pass configuration through environment variables — the twelve-factor way. Spring maps `SPRING_DATASOURCE_URL` to `spring.datasource.url` automatically.

```bash
docker run -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=prod \
  -e SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/shop \
  bytemart/shop:1.0
```

Production practices

- Wire container health checks to Actuator's **liveness/readiness** probes.
- Set JVM memory to respect the container's limits.
- Inject secrets from the platform, never bake them into the image.
- Tag images by version and deploy immutable artifacts.

© 2026 ByteMart Digital

Graceful shutdown

Enable graceful shutdown so in-flight requests finish before the app stops — essential for zero-downtime rolling deployments.

```text
server:
  shutdown: graceful
```

Ship the same thing everywhere

Containerising a Boot JAR gives you an identical, self-contained artifact from laptop to production. Configure via environment variables, wire health probes, handle secrets and shutdown properly — and deployments become predictable.

MODULE 8 · ADVANCED & DEPLOYMENT

CHAPTER 30

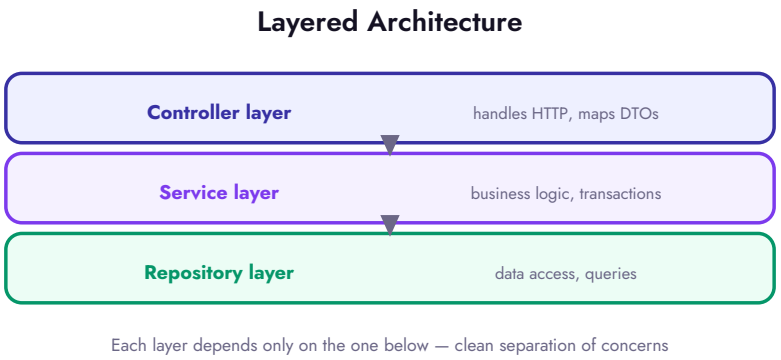
Best Practices & Architecture

Structuring a maintainable Spring Boot application and the habits that keep it healthy.

Everything in this guide comes together in how you structure a real application. Good Spring Boot code is layered, loosely coupled, well-configured, and observable — built so it stays easy to change as it grows.

Layered architecture

Separate responsibilities into clear layers, each depending only on the one below. Controllers handle HTTP, services hold business logic and transactions, repositories handle data.



Controller, service, and repository layers — each depends only on the one below.

Structure and coupling

- Keep **controllers thin** and business logic in services.
- Expose **DTOs**, never entities, at the API boundary.
- Depend on **interfaces**; inject via the constructor.
- Organise by **feature**, not just by technical layer, as apps grow.

Configuration and secrets

- Externalise config; bind it type-safely with `@ConfigurationProperties`.
- Use profiles for environments; activate `prod` explicitly.
- Inject secrets from the environment or a secrets manager.
- Fail fast on missing required configuration at startup.

Reliability and operability

- Validate input at the edge; handle errors in one place.
- Make endpoints and jobs idempotent where possible.
- Expose health and metrics; log structured, secret-free events.

© 2026 ByteMart Digital

- Test in layers: many unit tests, fewer integration and web tests.

TIP

Start simple. A well-layered monolith with clean boundaries serves most applications well — introduce complexity like extra modules or services only when a real need appears, not preemptively.

Layered, decoupled, observable

A healthy Spring Boot app keeps layers clean, exposes DTOs, externalises config, and is tested and observable. Master the fundamentals in this guide and apply them consistently — architecture is those habits, repeated.